Project Title: Trace-Driven Memory Cache Simulator with Custom Replacement Policy Evaluation

Team Members:

1. Aryan Sandal (2024CSB1103)
2. Moinak Goswami (2024CSB1225)

List of Features/Specifications:

- **Trace-Driven Execution:** The simulator will process memory access traces (instruction and data) generated using the Intel PIN tool.
- **Configurable Architecture:** Support for dynamic configuration of Cache Size, Block/Line Size, and Set Associativity (Direct-Mapped, N-way Set Associative, Fully Associative).
- **Standard Replacement Policies:** Implementation of baseline algorithms, including LRU (Least Recently Used), LFU (Least Frequently Used), and Random replacement.
- **Optimal Baseline:** Implementation of Belady's MIN algorithm to establish a theoretical upper bound for cache hit rates on given traces.
- **Custom Replacement Policy:** Design and implementation of a novel heuristic-based replacement policy aimed at outperforming standard LRU/LFU on specific workload patterns.
- **Performance Metrics Engine:** Tracking and reporting of Total Accesses, Hit/Miss Ratios, Evictions, and estimated Average Memory Access Time (AMAT).
- **Core Implementation:** The simulator will be written entirely from scratch in C++ to ensure low-overhead processing of massive memory traces.

Our tentative Implementation Plan:

- **Phase 1: Environment Setup & Trace Generation (Week 1)**
    - Configure the Intel PIN tool environment.
    - Modify PIN scripts to generate realistic memory read/write traces from standard benchmark programs.
- **Phase 2: Base Simulator Development (Week 1-2)**
    - Design the C++ class structures for Cache, Sets, and Blocks.
    - Implement trace parsing and the core memory addressing logic (Tag, Index, Offset extraction).
- **Phase 3: Standard and Optimal Policies (Week 2-3)**
    - Integrate LRU, LFU, and Random policies.
    - Implement Belady's MIN algorithm (requires preprocessing the trace to determine future access distances).
- **Phase 4: Custom Policy Design & Integration (Week 3-4)**
    - Develop the custom replacement algorithm.
    - Integrate the custom policy into the simulator framework.
- **Phase 5: Benchmarking & Analysis (Week 4)**
    - Run all policies against multiple workload traces.
    - Compile comparative results (Hit rates vs. Associativity vs. Policy) and finalize the project report.